

Prerequisites

Install the following R package:

- *camtrapR*
- *taxize*
- *overlap*

Install exiftool (external software, <https://exiftool.org>)

Install MerlinBird on your phone

Learning objectives

- Students are able to organise raw camera trap data using *camtrapR*
- Students are able to identify common British animals from camera trap images
- Students are able to run basic analyses using camera trap data

Camera trap practical

1. Import the camera trap images from the shared OneDrive to your computer. You have one station for your group, so you can divide the pictures amongst your group. Make sure to name the folders according to the station name. As station name, you can use the Data logger serial number, as used during the Epicollect form (<https://five.epicollect.net/project/silwood-park-ecological-data>). Make sure to copy the exact name from that form, as we will be using that later.
2. Camera traps typically do not give images informative names. We want image names to contain the image metadata, i.e. the station name, date, and time. The `imageRename()` function does exactly this, by using metadata to rename images, so that an image from station DL-06, taken at 17-10-2025 at 10:00 AM becomes 'DL-06__2025-10-17__10-00-00.JPG'. The function requires an input and output directory, like so:

```
imageRename(inDir = "path_to_your_input_folder",  
            outDir = "path_to_your_output_folder", keepCameraSubfolders = F,  
            hasCameraFolders = F, copyImages = T)
```

We have `keepCameraSubfolders = F` and `hasCameraFolders = F`, because we run it on a single subfolder. If you were to have a large dataset, containing multiple camera stations and/or multiple cameras per station, this would be set to T. Rename the camera trap images using the `imageRename()` function, so that they contain this information.

3. To sort the images, we will create subfolders for each species, within the camera trap station subfolder, so that you have “*station_subfolder/species_subfolder*”. Because multiple people will be ID’ing animals, it is especially important to be consistent with species names. To avoid confusion, we use scientific species names. CamtrapR uses the [ITIS](#) nomenclature, so you can check species names through their online database. Folder names should follow the format *genus_species*, for example *Argusianus_argus* (note the underscore between the genus and species name!) for the Great Argus pheasant. Sort the images, creating new subfolders as you encounter new species.
4. To make sure you have named species consistently and correctly, you can use the `checkSpeciesNames()` function. It requires a species name as input, and then check it for matches in the IT IS database. You can run it like this:

```
checkSpeciesNames(speciesNames = "Argusianus argus",
searchtype = 'scientific')
```

This will return this dataframe in your console:

	tsn	user_name	scientificName	commonName	authorship	rankname
1	176111	Argusianus argus	Argusianus argus	Argus géant/Argos real/argos-real/Great Argus	(Linnaeus, 1766)	Species
				itis_url	taxon_status	
1	https://www.itis.gov/servlet/SingleRpt/SingleRpt?search_topic=TSN&search_value=176111				valid	

If common name matches your species of interest, and the taxon status is valid, you have named the folder correctly. Do this for each species you identified. Copy the folder names you used, and make sure to replace the underscore between the genus and species name with a space.

5. After you have gone through all images and identified all species, create a table of the camera trap records using `recordTable()`.

```
sp_table = recordTable(inDir = "/your_parent_folder",
IDfrom = "directory", minDeltaTime = 30,
deltaTimeComparedTo = "lastIndependentRecord")
```

6. Now export the recordTable, so you can upload it to the shared Box. Name it according to the name of the original folder you worked on. Make sure to end the filename with `.csv`.

```
library(utils)
write.csv(sp_table,
"path_to_your_directory/sp_table_name_of_your_folder.csv")
```

7. Upload the table to the folder `Camera_Traps/2025/sp_table_name_of_your_folder` in Box.

8. Once everyone has uploaded the `sp_tables`, download all the `sp_tables`. Read all the `sp_tables` into your RStudio. First, make a list of all tables in your folder, and then read them.

```
list.of.files = list.dirs("path_to_your_table_folder")
sp_table = read.csv(list.of.files[1])
for (i in 2:length(list.of.files)){
  temp_table = read.csv(list.of.files[i])
  sp_table = rbind(sp_table, temp_table)
}
```

9. Now that you have the camera trap data, we can start looking for patterns. Create a table of all the camera trap records using `recordTable()` :
What do the `minDeltaTime` and `deltaTimeComparedTo` arguments mean, and why these are important when analysing camera trap data?

10. One aspect of species' behaviour we can investigate with camera trap data are their activity patterns. Use the `activityDensity()` function to plot species' activity curves:

```
activityDensity(sp_table, allSpecies = TRUE)
```

You can navigate through the different species' graphs by pressing the backwards and forwards bottom in RStudio's viewer window. What does the density on the y-axis mean?

11. Another way to use camera trap data is comparing the activity patterns of two different species. Pick two species and do this with `activityOverlap()` :

```
activityOverlap(sp_table,
speciesA = "species_of_interest_A", speciesB = "species_of_interest_B")
```

What kind research questions would you be able to investigate with these graphs?

12. Camera trap data can also help us compare species richness between different stations, seasons, or habitat types, to name a few. Using the table you made with `recordTable()`, create a new table with the species richness per site.
13. Use this data to compare and plot the species richness between the different habitat types at Silwood Park. To do this, we'll first need to merge our metadata table from Epicollect with the species richness data. The 'Station' column in your detection table should correspond to the "Data logger serial number" column in the Epicollect. After

loading the Epicollect form into R, merge the two dataframes using R's `merge()` function, like so:

```
detection_dataframe = merge(species_richness_dataframe,  
epicollect_dataframe,  
by=c("column_name_detection_dataframe"="column_name_epicollect"))
```

After you have merged the dataframes, compare the species richness between the different habitat types, and make a simple plot to visualise it.